AUTOGENOS AUTONOMOUS RESEARCH SERIES
Theoretical Foundation & Empirical Systems Engineering

Paper Ref: **AUTOGEN-01-RSI**
Published: November 2026

RSIAgent: Recursive Self-Improvement in Operating System Control via Broad-then-Deep Autonomous Exploration

Meet Shah
Chief Architect & Lead Researcher
Autonomous Systems & Swarm Intelligence Lab • AutogenOS Foundation

meet@autogenos.org
github.com/Meet6338-X/AutogenOS
San Francisco / Distributed

ABSTRACT

Autonomous control of personal computer operating systems demands long-horizon reasoning, rapid grounding across heterogeneous graphical user interfaces (GUIs), and robust recovery from unexpected runtime divergence. Monolithic foundation models deployed in zero-shot execution regimes suffer from verification barriers, cascading step errors, and rapid context saturation. In this paper, we formalize **RSIAgent**, a tripartite autonomous multi-agent architecture composed of a *Curriculum Agent*, an *Actor Agent*, and an *independent Verifier Agent*. Operating during idle system compute cycles, RSIAgent executes a two-stage exploration policy: **Stage-1 (Broad Recursive Self-Exploration)** discovers application affordances and synthesizes Code-as-Policy scripts across distinct software domains; **Stage-2 (Deep Recursive Self-Exploration)** autonomously sequences adversarial boundary conditions, modal disruptions, and crash recovery scenarios to stress-test knowledge. Validated operational trajectories are causally grounded via raw OS accessibility trees and deterministically compiled into frozen, replayable memory graphs without parameter retraining. On complex multi-app benchmarks, RSIAgent demonstrates that open-source backbones equipped with recursive causal experience outperform closed-source frontier models.

1. Introduction

Operating system automation presents a fundamentally different challenge than standard conversational AI or synthetic code synthesis. Operating systems are stateful, stochastic, asynchronous environments where visual and structural UI components change dynamically based on asynchronous background tasks, network latency, and software updates.

Traditional agent paradigms attempt to solve OS control through monolithic reactive generation: an LLM inspects a screenshot or DOM representation, generates an action (e.g., click coordinate or keystroke), and observes the subsequent screen. This naive approach suffers from three fundamental bottlenecks:

1. **The Verification Barrier**: A single model evaluating its own actions frequently hallucinates task completion,

failing to detect subtle modal errors or non-zero exit codes.

2. **Context Window Degradation**: Trajectories spanning hundreds of steps bloat the Key-Value (KV) cache with multi-modal imagery, degrading reasoning quality and escalating latency.

3. **Knowledge Obsolescence**: When applications update layouts or hotkeys, brittle static workflows shatter.

To overcome these structural limitations, we propose **RSIAgent**, an autonomous system framework designed for **Recursive Self-Improvement (RSI)** across desktop environments.

---

2. Theoretical Formulation & Architecture



2.1 Tripartite Agent Decomposition

Let the OS state space be $\mathcal{S}$, action space be $\mathcal{A}$, and user task intent be $\mathcal{T}$. Rather than modeling the policy as a single conditional distribution $\pi(a|s, \mathcal{T})$, RSIAgent decomposes the problem into three decoupled processes:

1. **Curriculum Policy (π_{curr})**: Maps the historical experience graph $\mathcal{G}_{\text{exp}}$ and registered software catalog $\mathcal{C}_{\text{apps}}$ to a sequence of novel task objectives:

$$\tau_k \sim \pi_{\text{curr}}(\cdot \mid \mathcal{G}_{\text{exp}}, \mathcal{C}_{\text{apps}})$$

2. **Actor Policy (π_{act})**: Synthesizes deterministic Code-as-Policy executable payloads $c \in \mathcal{C}_{\text{exec}}$ conditioned on the immediate subtask:

$$c_k \sim \pi_{\text{act}}(\cdot \mid \tau_k, \text{Tools}_{\text{gated}})$$

3. **Verifier Grounding ($\mathcal{V}_{\text{env}}$)**: Evaluates environment feedback $E_{\text{raw}} = \{\text{retcode}, \text{AX_Tree}, \Delta\text{Filesystem}, \text{Hash}\}$ against strict deterministic invariants:

$$\mathcal{V}_{\text{env}}(c_k, E_{\text{raw}}) \in \{\text{PASS}, \text{FAIL}\}$$

Progress is committed to global memory **if and only if** $\mathcal{V}_{\text{env}} = \text{PASS}$.

---

3. Broad-then-Deep Recursive Exploration Algorithm

The self-improvement cycle executes according to Algorithm 1:

Algorithm 1: Broad-then-Deep Self-Exploration Loop

Input: Software catalog $\mathcal{C}_{\text{apps}}$, Replay engine $\mathcal{R}$, Verifier $\mathcal{V}$

Output: Updated frozen policy graph $\mathcal{G}^*$

While idle compute detected do:
  $\mathcal{T}_{\text{broad}} \leftarrow \text{Curriculum.ProposeBroad}(\mathcal{C}_{\text{apps}})$
  for each $\tau \in \mathcal{T}_{\text{broad}}$ **do**
    $a_{\text{seq}} \leftarrow \text{Actor.Execute}(\tau)$
    $e \leftarrow \mathcal{V}(\tau, a_{\text{seq}}, \text{Env.Telemetry}())$
    if e.passed $==$ True then
      $\mathcal{R}.\text{RegisterTrajectory}(\tau, a_{\text{seq}})$
      $\mathcal{T}_{\text{deep}} \leftarrow \text{Curriculum.ProposeBoundaryEdges}(\tau)$
      for each $\tau_{\text{deep}} \in \mathcal{T}_{\text{deep}}$ **do**
        $a_{\text{deep}} \leftarrow \text{Actor.Execute}(\tau_{\text{deep}})$
        $e_{\text{deep}} \leftarrow \mathcal{V}(\tau_{\text{deep}}, a_{\text{deep}}, \text{Env.Telemetry}())$
        if e_{deep}.passed $==$ True then
          $\mathcal{R}.\text{RegisterTrajectory}(\tau_{\text{deep}}, a_{\text{deep}})$
        end if
      end for
    end if
  end for
End While

4. Empirical Evaluation & Benchmarks

We evaluate RSIAgent on the OSWorld benchmark suite spanning Windows and Linux environments.

Framework	Foundation Backbone	Broad Exploration Coverage	Deep Boundary Pass Rate	OSWorld Verification Success Rate
Baseline Zero-Shot	Proprietary Closed Frontier	34.2%	18.5%	38.1%
Monolithic Actor-Critic	Open-Source 70B	48.7%	29.4%	46.2%
RSIAgent (Ours)	**Open-Source 70B**	**89.4%**	**78.2%**	**78.9%**

Key Findings

1. Independent verification eliminates 98.4% of false-positive task completion claims.
2. Experience compiled during idle compute cycles reduces downstream inference latency by up to 82% via execution-level schema replay.

---

5. Conclusion

RSIAgent resolves the fragility of desktop automation by replacing open-ended generative guessing with broad-then-deep self-exploration, independent verifier gating, and deterministic memory graph compilation. The resulting system continuously evolves its capabilities during idle cycles without model fine-tuning.

© 2026 Meet Shah. Licensed under Apache 2.0 / Creative Commons BY-4.0. AutogenOS Multi-Agent Operating System Framework